SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY

TECHNICAL UNIVERSITY OF MUNICH

Master's Thesis in Computer Science

Dynamic Resource Management in OpenMP

Darius Augsburger

SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY

TECHNICAL UNIVERSITY OF MUNICH

Master's Thesis in Computer Science

Dynamic Resource Management in OpenMP

Dynamische Ressourcenverwaltung in OpenMP

Author:	Darius Augsburger
Examiner:	Prof. Dr. Martin Schulz
Supervisor:	Prof. Dr. Martin Schreiber
Submission Date:	October 15, 2026

I confirm that this master's thesis is my own work and I have documented all sources and material used.

Munich, October 15, 2026 Darius Augsburger

Acknowledgments

I would like to express my sincere gratitude to my main advisor, Prof. Dr. Martin Schreiber from the Technical University of Munich (TUM), for his continuous guidance, insightful discussions, and constructive feedback throughout the course of this thesis. His expertise in parallel programming and runtime systems, as well as his encouragement to critically analyze both practical implementations and standardization aspects, significantly shaped the direction and quality of this work.

I would also like to thank my advisors from RWTH Aachen University for their valuable input and discussions on parallel runtime systems and resource management, which contributed important perspectives to this thesis.

Furthermore, I am grateful to the members of the research group for the stimulating environment and informal exchanges that helped deepen my understanding of OpenMP runtimes and dynamic resource management.

Above all, I would like to express my deepest gratitude to my wife. Her constant support, patience, and understanding made it possible for me to pursue this work alongside a full-time job, the responsibilities of running my own company, and the completion of this master's thesis in parallel. Without her encouragement and support, this work would not have been possible.

Finally, I would like to thank my family and friends for their continued support throughout my studies.

Abstract

This thesis addresses the implementation of dynamic resource management in OpenMP, focusing on the allocation and management of CPU cores—and potentially GPUs—during program execution. Although the `OMP_DYNAMIC` flag has been part of the OpenMP standard for over two decades, it has not been fully realized in practice. The work extends parallel regions in LLVM/OpenMP to enable the addition and removal of threads at runtime, thereby improving flexibility and resource utilization. An in-depth analysis of existing thread pool and worker thread mechanisms in libomp is conducted, and the feasibility of integrating a resource manager communication structure is explored. Compatibility with the OpenMP standard is evaluated, with necessary adaptations or deprecations proposed. The integration is designed to be accessible via environment variables or API. The effectiveness of the approach is assessed through comprehensive benchmarks, focusing on performance, overhead, and compatibility.

Contents

Acknowledgments **iii**

Abstract **iv**

1 Introduction **1**

2 Background and Foundations **2**
2.1 OpenMP Execution and Runtime Model 2
2.2 Thread Lifecycle and Thread Pools in OpenMP Runtimes 2
2.3 Dynamic Resource Management Concepts 3
2.4 Related Work and Existing Approaches 4

3 Empirical Analysis of the LLVM OpenMP Runtime **5**
3.1 Experimental Setup 5
3.2 Thread Creation, Reuse, and Pool Behavior 5
3.3 Behavior Across Multiple Parallel Regions 6
3.4 Implications for Dynamic Resource Management 6
3.5 Overview of LLVM 7
3.6 Dynamic Resource Management in the OpenMP Runtime 7

4 Design Space for Dynamic Resource Management in OpenMP **10**
4.1 Constraints Imposed by the OpenMP Standard 10
4.2 Internal vs. External Resource Management Approaches 10
4.3 Control Granularity and Decision Points 11
4.4 Policy Design Considerations 11

5 Evaluation of Permit-Based Resource Arbitration using TCM **13**
5.1 TCM Architecture 13
5.2 Integration into OpenMP 14
5.3 Limitations with Multi-Process Synchronization 14
5.4 Conceptual Influence on This Work 15
5.5 Positioning Relative to This Thesis 15

6 Free Agent Threads in OpenMP **16**

7 Custom OpenMP Resource Manager **18**
7.1 Architecture and Integration 18
7.2 Implementation Challenges 19
7.2.1 Fair-Share Scheduler Override 19
7.2.2 LLVM Runtime Load-Average Heuristic Override 19
7.2.3 Sequential Benchmark Iterations 20
7.3 CPU Affinity and Exclusive Partitioning 20

8 Evaluation and Benchmarking **22**
8.1 Experimental Setup 22
8.1.1 Hardware Platforms 22
8.1.2 Evaluation Methodology 22
8.2 NAS Parallel Benchmarks 23
8.2.1 Kernel Suite 23
8.2.2 Selection Rationale 23
8.2.3 EP (Embarrassingly Parallel) 23
8.2.4 CG (Conjugate Gradient) 24
8.2.5 FT (Fast Fourier Transform) 24
8.3 Single-Process Baseline 24
8.4 DRM Evaluation: Concurrent Two-Process Results 24
8.4.1 EP (Embarrassingly Parallel) 25
8.4.2 FT (3-D FFT, Memory-Bandwidth Bound) 25
8.4.3 CG (Conjugate Gradient, Irregular Memory Access) 26
8.5 Key Findings 26

9 Implications and Standardization Proposal for OpenMP **28**

10 Summary **29**

Abbreviations **30**

Bibliography **31**

1 Introduction

Modern high-performance and parallel applications increasingly face dynamic and heterogeneous execution environments, where the optimal allocation of computational resources may change over time. While OpenMP [DM98] provides a high-level programming model for shared-memory parallelism, its execution model largely assumes static or monotonically increasing resource allocation within a process.

Although the OpenMP standard has included the `OMP_DYNAMIC` mechanism for more than two decades [Ope24], dynamic adjustment of computational resources has not been fully realized in practice. In particular, current OpenMP runtimes typically rely on fixed or lazily growing thread pools, without providing standardized mechanisms to dynamically adapt the number of active threads across parallel regions or in response to external resource constraints.

The goal of this thesis is to investigate and prototype dynamic resource management for OpenMP, with a primary focus on CPU cores and an outlook towards accelerator resources such as GPUs. The work is based on an in-depth analysis of the LLVM OpenMP runtime (libomp) [], including the lifecycle of worker threads, thread pool behavior, and relevant control points within the runtime.

Building on this analysis, different approaches to dynamic resource management are explored. This includes the integration of an external resource manager via a communication mechanism as a proof of concept, as well as the design and implementation of a custom, runtime-internal resource manager supporting multiple allocation strategies. The feasibility, overhead, and limitations of these approaches are evaluated with respect to the OpenMP execution model and standard compliance.

Finally, the proposed mechanisms are evaluated using microbenchmarks and application benchmarks to assess performance impact, overhead, and compatibility. Based on the experimental findings, this thesis discusses implications for OpenMP runtimes and derives a proposal for standardizing dynamic resource management interfaces in future versions of the OpenMP standard.

2 Background and Foundations

This section introduces the fundamental concepts required to understand dynamic resource management in OpenMP runtimes. It covers the OpenMP execution and runtime model, the lifecycle of threads and the use of thread pools in OpenMP implementations, core concepts of dynamic resource management, and an overview of related work and existing approaches.

2.1 OpenMP Execution and Runtime Model

OpenMP is a directive-based programming model for shared-memory parallelism that follows a fork–join execution paradigm [DM98]. A program starts execution in a single master thread. When a parallel region is encountered, the master thread forks a team of worker threads, which execute the parallel region concurrently. At the end of the region, the worker threads synchronize and join back into a single thread of execution.

The OpenMP standard defines the semantics of parallel regions, synchronization constructs, and work-sharing constructs, while leaving many implementation details to the runtime [Ope24]. The number of threads used in a parallel region can be influenced by clauses such as `num_threads`, internal control variables (ICVs), and environment variables such as `OMP_NUM_THREADS` and `OMP_DYNAMIC`. However, the standard does not prescribe how threads are created, managed, or reused internally by a runtime.

Crucially, OpenMP does not allow changes to the team size while a parallel region is active [Ope24]. Decisions about the number of threads must therefore be made at well-defined synchronization points, most notably at the fork barrier when entering a parallel region. This restriction strongly influences the design space for dynamic resource management in OpenMP.

2.2 Thread Lifecycle and Thread Pools in OpenMP Runtimes

Most modern OpenMP runtimes, including the LLVM OpenMP runtime [], implement thread management using thread pools rather than creating and destroying threads for each parallel region. Threads are typically created lazily, often during the first parallel region that requires them, and are subsequently reused across multiple regions.

Once created, worker threads may transition between active and inactive states. Inactive threads may either spin-wait, sleep, or remain parked depending on runtime configuration and environment variables such as `OMP_WAIT_POLICY` [Ope24]. Terminating threads is usually avoided due to the high overhead associated with thread creation, and thread pools often grow monotonically up to the maximum number of threads ever required by the application. Threads are commonly only reclaimed when the runtime shuts down.

These behaviors are considered quality-of-implementation aspects rather than standardized features. As a result, different OpenMP runtimes may exhibit different strategies with respect to thread reuse, sleep states, and pool growth. Understanding these implementation choices is essential for assessing the feasibility and cost of dynamic resource management mechanisms.

2.3 Dynamic Resource Management Concepts

Dynamic resource management refers to the ability of a system to adapt its use of computational resources at runtime in response to changing workloads, system load, or external constraints. In the context of parallel applications, this often involves adjusting the degree of parallelism, such as the number of threads or cores used.

Two closely related concepts are commonly distinguished [FR96]. Elasticity refers to the ability to adjust resource usage between parallel regions or phases of execution, while malleability describes the ability to change resource allocation during an active parallel phase [Ali+22]. Due to OpenMP's execution model and semantic constraints, this thesis focuses on elastic resource management rather than malleable execution.

Dynamic resource management is often motivated by oversubscription scenarios [Ian+10], multi-application workloads, or co-execution with other parallel libraries. Performance models such as Amdahl's law [Amd67] and Gustafson's law [Gus88] provide useful intuition for understanding how changes in parallelism affect overall performance, and they inform the design and evaluation of resource management policies.

Resource management can be implemented at different levels, ranging from operating-system mechanisms such as CPU affinity [Eic+12] and control groups [Men04] to runtime-level decisions within a parallel programming framework. This work focuses on runtime-level resource management, as it allows decisions to be made with knowledge of the OpenMP execution structure while remaining portable across platforms.

2.4 Related Work and Existing Approaches

Several approaches related to resource management in OpenMP and parallel runtimes have been proposed. The OpenMP Tools (OMPT) [Eic+13] and Debug (OMPD) [Pro+15] interfaces provide standardized mechanisms for observing runtime behavior, but they are explicitly designed for monitoring and debugging rather than active control of resource allocation.

External resource managers, such as operating-system-level mechanisms or runtime coordination frameworks, have been explored to manage resource sharing between multiple parallel applications or libraries. Examples include control-group-based approaches [Men04] and coordination frameworks such as the Thread Composability Manager (TCM) [FVI]. While these systems aim to provide global resource coordination, their integration with OpenMP runtimes is limited by the lack of standardized control interfaces.

Previous research has investigated dynamic resource allocation, adaptive parallelism, and elastic execution in high-performance computing [Ali+22]. Approaches such as DROM [DAm+18] and dynamic load balancing via the LeWI algorithm [GLC14] demonstrate that runtime-level malleability and thread count adaptation can yield measurable performance improvements. Complementary work explores dynamic core reassignment between MPI ranks parallelized with TBB [SW18] and cluster-based resource management for adaptive grid simulations [Sch]. However, many of these approaches rely on application-specific modifications, custom runtimes, or assumptions that are not directly compatible with existing OpenMP implementations.

Overall, existing work highlights both the potential benefits of dynamic resource management and the limitations imposed by current OpenMP standards and runtime interfaces. These gaps motivate the exploration of runtime-integrated mechanisms and the development of proposals for standardized support for dynamic resource management in OpenMP.

3 Empirical Analysis of the LLVM OpenMP Runtime

This chapter presents an empirical analysis of the LLVM OpenMP runtime (`libomp`) with respect to thread management and resource usage. The objective is to understand how threads are created, reused, and managed across parallel regions in practice, and to identify the constraints and control points relevant for dynamic resource management.

3.1 Experimental Setup

All experiments were conducted using the LLVM OpenMP runtime (`libomp`) as provided by the Clang/LLVM toolchain. Unless stated otherwise, OpenMP programs were compiled using Clang with OpenMP support enabled. The analysis focuses on CPU-based execution and shared-memory parallelism.

To observe runtime behavior, OpenMP environment variables were used to control thread placement, affinity, and runtime verbosity. Runtime debug output as well as operating-system-level inspection tools were employed to trace thread creation, reuse, and lifecycle events. Simple OpenMP microbenchmarks consisting of multiple parallel regions with varying thread counts were used to isolate runtime behavior from application-specific effects.

3.2 Thread Creation, Reuse, and Pool Behavior

An initial set of experiments investigates how worker threads are created and managed by the OpenMP runtime. When a parallel region is encountered for the first time, the runtime creates a team of worker threads according to the requested team size. Thread creation is typically performed lazily, meaning that threads are spawned only when they are first required.

Once created, worker threads are not terminated at the end of a parallel region. Instead, they transition into an inactive state and remain part of an internal thread pool. Subsequent parallel regions may reuse these existing threads, avoiding the overhead associated with repeated thread creation. The thread pool grows monotonically

to accommodate the maximum number of threads ever requested during program execution.

Thread termination is generally deferred until the runtime shuts down at program termination. This behavior reflects a quality-of-implementation decision that prioritizes low overhead and fast re-entry into parallel regions over aggressive resource reclamation.

3.3 Behavior Across Multiple Parallel Regions

To further analyze runtime behavior, multiple parallel regions with varying team sizes were executed sequentially within a single process. The experiments show that reducing the requested number of threads in later parallel regions does not lead to a reduction in the size of the thread pool. Threads that are no longer required remain allocated but inactive.

Operating-system-level inspection confirms that threads persist across parallel regions even when the active team size decreases. The runtime distinguishes between active and inactive threads, but does not dynamically shrink the pool. As a result, resource usage in terms of allocated threads reflects the maximum parallelism reached during execution rather than the current requirements of individual regions.

These observations are consistent with the OpenMP standard, which specifies the semantics of parallel regions but leaves internal thread management strategies to the runtime implementation.

3.4 Implications for Dynamic Resource Management

The empirical findings have several important implications for dynamic resource management in OpenMP runtimes. First, the fork barrier at the beginning of a parallel region represents the primary safe point at which decisions about resource allocation can be made. The OpenMP execution model does not permit changes to the number of threads during the execution of a parallel region.

Second, since thread pools are typically persistent and grow monotonically, dynamic resource management should focus on controlling the number of active threads rather than the creation or destruction of threads. Mechanisms such as thread sleeping, parking, or selective activation are therefore more practical than aggressive thread termination.

Finally, the absence of standardized interfaces for influencing runtime decisions highlights a gap between what is semantically allowed by the OpenMP standard and what is practically supported by existing runtimes. These observations motivate the

exploration of runtime-integrated resource management mechanisms and inform the design space discussed in subsequent chapters.

3.5 Overview of LLVM

The LLVM project (originally "Low Level Virtual Machine") has evolved into a widely used compiler infrastructure that provides a modular and reusable toolchain for programming languages [LA04]. Its core concept is the separation of the compilation process into three main stages: *front-end, optimizer, and back-end*.

In the front-end, source code written in high-level languages (e.g., C/C++ via Clang, Swift, or Rust) is translated into a common intermediate representation (LLVM IR). This IR is a low-level, static single assignment (SSA)-based code representation that resembles assembly language while remaining platform-independent. The use of a uniform IR enables language front-ends to share the same optimization and code generation infrastructure.

The middle layer, often referred to as the optimizer, applies a configurable pipeline of transformation passes to the LLVM IR. These passes perform a wide range of optimizations such as constant propagation, dead-code elimination, inlining, and loop unrolling. Since the IR is target-independent, the majority of these optimizations can be applied consistently across different architectures.

In the final stage, the back-end lowers the optimized IR into target-specific machine code. This involves instruction selection, register allocation, instruction scheduling, and code emission. LLVM supports a wide variety of architectures (x86, ARM, RISC-V, PowerPC, among others), which allows the same high-level source code to be compiled efficiently for diverse platforms [].

Due to this modular architecture, LLVM has become the foundation of many modern programming languages and toolchains. Beyond traditional compilation, it is also widely employed in just-in-time (JIT) compilation, GPU programming, and static program analysis, making it a central component in both academic research and industrial applications.

3.6 Dynamic Resource Management in the OpenMP Runtime

OpenMP parallel regions in Clang/LLVM are ultimately lowered into calls to the OpenMP runtime library (`libomp`) []. A central entry point is the function `__kmpc_fork_call`, which is generated by Clang whenever a `#pragma omp parallel` directive is encountered. This function is responsible for initializing a new team of threads and invoking the corresponding microtask function that represents the user's parallel region.

In the current implementation, the number of threads participating in a team is derived from environment variables (e.g., `OMP_NUM_THREADS`), runtime settings, or heuristics within the runtime system itself. While this approach is sufficient for static or semi-static resource allocation, it lacks the ability to adapt dynamically to system load, heterogeneous architectures, or energy constraints [].

To enable dynamic resource management, the OpenMP runtime is the most effective integration point. Unlike modifications at the compiler or IR level, which would require changes to Clang's front-end or the LLVM optimizer pipeline, runtime-level adaptation allows resource allocation policies to be evaluated and enforced at execution time. This is critical for scenarios where system load, available devices, or power budgets fluctuate dynamically.

A practical design involves introducing a *Resource Manager* as an intermediate layer within the runtime. The manager intercepts team creation in `__kmp_fork_call` and determines the appropriate resource allocation strategy. Conceptually, it can expose the following interface:

```
class ResourceManager {
public:
    static ResourceManager& getInstance();

    int decide_num_threads(const ident_t *loc, int requested);
    void assign_resources(int team_id, int num_threads);
    void release_resources(int team_id);
};
```

This manager is invoked during the fork process:

```
int requested = num_threads_from_env_or_runtime();
int nthreads = ResourceManager::getInstance().decide_num_threads(loc, requested);

__kmp_allocate_team(gtid, nthreads);
ResourceManager::getInstance().assign_resources(team_id, nthreads);
```

Such a design provides a central hook for implementing different resource management policies, including:

- System-load aware policies, where the number of threads adapts to CPU utilization.
- NUMA-aware placement, where threads are bound to the closest memory domains.

- Hybrid CPU/GPU distribution, in combination with `libomptarget` for accelerator offloading.
- Energy-aware scheduling, where parallelism is reduced under thermal or power limits.

This approach has several advantages for research and evaluation. First, it separates policy from mechanism, allowing new allocation strategies to be plugged into the runtime without modifying the compiler. Second, it provides fine-grained control at the point of execution, enabling reproducible experiments with dynamic scheduling, resource elasticity, and heterogeneous execution. Finally, it extends OpenMP's otherwise static resource management model, aligning it with modern trends in adaptive runtime systems and performance-portable computing [Ali+22].

4 Design Space for Dynamic Resource Management in OpenMP

This chapter discusses the design space for dynamic resource management in OpenMP, based on the execution model defined by the OpenMP standard and the empirical observations of the LLVM OpenMP runtime. The goal is to identify feasible design choices, constraints, and trade-offs for integrating dynamic resource management mechanisms into OpenMP runtimes.

4.1 Constraints Imposed by the OpenMP Standard

The OpenMP standard defines a fork–join execution model in which parallel regions are executed by a team of threads that is created at the beginning of the region and joined at its end [Ope24]. While the standard allows the number of threads to vary between parallel regions, it does not permit changes to the team size during the execution of a parallel region.

This restriction imposes a fundamental constraint on dynamic resource management in OpenMP. Any adaptation of resource allocation must occur at well-defined synchronization points, most notably at the fork barrier when entering a parallel region. As a result, resource management in OpenMP is inherently elastic rather than malleable [FR96].

In addition, the standard defines a precedence hierarchy for determining the number of threads, including explicit `num_threads` clauses, internal control variables, and environment variables [Ope24]. A dynamic resource management mechanism must respect this hierarchy to remain compliant with the OpenMP specification.

4.2 Internal vs. External Resource Management Approaches

Dynamic resource management can be implemented either within the OpenMP runtime itself or by an external entity that coordinates resource usage across multiple applications or runtimes.

An internal resource management approach integrates decision-making logic directly into the OpenMP runtime. This allows the runtime to make resource allocation decisions with direct access to execution context, such as the structure of parallel regions and runtime state. Internal approaches benefit from low communication overhead and tight integration with runtime mechanisms, but they are limited to the scope of a single runtime instance.

External resource management approaches delegate resource allocation decisions to a separate process or service. Such approaches enable coordination across multiple applications or parallel libraries, potentially allowing for system-wide resource optimization. Similar ideas have been explored at the MPI level, where simulation layers can model the effect of dynamically varying compute resources without modifying the application [Fec]. However, external approaches introduce additional communication overhead and require well-defined interfaces between the runtime and the external manager. The lack of standardized control interfaces in OpenMP runtimes currently limits the applicability of external approaches.

4.3 Control Granularity and Decision Points

A key design decision concerns the granularity at which resource management decisions are made. In OpenMP, the natural unit of control is the parallel region, as resource allocation decisions can only be applied when a new team of threads is formed.

Resource management decisions may therefore be applied on a per-region basis, allowing different parallel regions within the same application to use different levels of parallelism. Finer-grained control, such as adapting resource allocation within a region, is not supported by the OpenMP execution model.

Another important consideration is the frequency of decision-making. Re-evaluating resource allocation at every parallel region provides maximum flexibility but may introduce overhead. Alternatively, decisions can be amortized across multiple regions or limited to phases of execution to reduce overhead.

4.4 Policy Design Considerations

The design of resource management policies involves balancing responsiveness, overhead, and predictability. Simple policies, such as fixed or staged allocation strategies, offer low overhead and deterministic behavior but may fail to adapt to dynamic workloads. More adaptive policies can respond to changes in system load or application behavior, but they require additional monitoring and may introduce decision latency.

Performance models such as Amdahl's law [Amd67] and Gustafson's law [Gus88] provide useful guidance for policy design by characterizing the relationship between parallelism and performance. These models can inform decisions about when increasing or decreasing the number of threads is likely to be beneficial.

Finally, practical policy design must account for runtime-specific considerations, including thread pool behavior, thread sleep states [], and affinity management [Eic+12]. Policies that align with these implementation characteristics are more likely to achieve effective and low-overhead dynamic resource management.

5 Evaluation of Permit-Based Resource Arbitration using TCM

Modern high-performance applications frequently combine multiple parallel programming models within a single process. Typical examples include OpenMP combined with task-based runtimes such as oneTBB or numerical libraries internally parallelized using OpenMP. While each runtime independently aims to maximize hardware utilization, the coexistence of multiple thread pools often leads to oversubscription of CPU resources. This phenomenon is commonly referred to as a performance composability problem.

Fedotov et al. introduce the Thread Composability Manager (TCM) as a shared CPU resource coordinator that arbitrates hardware resources among parallel runtimes within a single process address space [FVI]. Instead of directly managing threads, TCM grants resource permits to participating runtimes and may renegotiate these permits dynamically.

Although TCM is not designed as a cluster-wide or multi-process resource manager, its architectural concepts provide valuable insight into dynamic resource management strategies that are relevant for OpenMP runtime adaptation. In this section, TCM is analyzed as a proof of concept for permit-based resource arbitration and positioned relative to the objectives of this thesis.

5.1 TCM Architecture

TCM is implemented as a process-wide singleton that coordinates participating runtimes via an explicit negotiation protocol [FVI]. Each runtime connects to TCM and requests a resource permit, which describes a subset of available hardware resources. A permit may include:

- Minimum and maximum concurrency levels
- CPU masks or NUMA node constraints
- Core-type constraints (e.g., performance or efficiency cores)
- Threads-per-core limits

- Priority information

Permits transition between different states such as `void`, `inactive`, `idle`, `pending`, and `active`. The runtime is responsible for mapping its internal thread pool onto the resources granted by an active permit. Importantly, TCM does not manage threads directly. It relies on the cooperation of integrated runtimes, which must respect granted permits and adjust their internal scheduling behavior accordingly.

5.2 Integration into OpenMP

In the TCM case study, an OpenMP team corresponds to a resource permit. At the fork barrier of a parallel region, the OpenMP runtime requests or activates a permit; at the join barrier, the permit transitions to an idle or inactive state [FVI]. However, a key limitation arises from the OpenMP execution model. Since, once a parallel region becomes active, its team size cannot be renegotiated until the region completes, OpenMP permits are treated as non-negotiable while active. This rigidity significantly restricts fine-grained dynamic adaptation. In contrast, task-based runtimes such as oneTBB allow threads to join or leave execution arenas dynamically, enabling mid-execution renegotiation. This observation highlights an important structural limitation of OpenMP's traditional fork-join model and directly motivates the research direction pursued in this thesis.

5.3 Limitations with Multi-Process Synchronization

TCM operates strictly within a single process address space. It does not coordinate:

- Multiple MPI ranks on the same node
- Independent processes sharing a NUMA domain
- Cluster-level job scheduling decisions
- System-wide resource redistribution

For hybrid HPC workloads (e.g., MPI + OpenMP), contention frequently occurs across process boundaries rather than solely between runtimes within one process. In such scenarios, a purely intra-process solution is insufficient.

Furthermore, TCM assumes that all participating runtimes are permit-aware and cooperative. It does not enforce resource constraints at the operating system level, nor does it integrate with external resource managers such as Slurm. Therefore, while TCM effectively addresses intra-process composability, it cannot serve as a complete solution for dynamic resource management in multi-process or cluster-scale environments.

5.4 Conceptual Influence on This Work

Despite its limitations, TCM introduces several architectural concepts that are relevant for this thesis:

1. **Permit-based abstraction.** Modeling hardware allocation as a negotiable entity simplifies reasoning about concurrency and ownership of CPU resources.
2. **Separation of arbitration and execution.** TCM separates policy decisions from thread pool mechanics. This separation inspires the design principle adopted in this thesis, where monitoring, decision logic, and enforcement are clearly modularized.
3. **Strategy-dependent performance behavior.** TCM evaluates multiple adaptive strategies and demonstrates that no single resource management strategy performs optimally across all workload compositions [FVI]. This finding reinforces the need for workload-aware dynamic scaling mechanisms.
4. **Importance of runtime compliance.** Experimental results in the TCM study show that incomplete runtime integration limits achievable performance gains. This motivates implementing dynamic resource management directly within the OpenMP runtime rather than relying solely on external coordination.

5.5 Positioning Relative to This Thesis

TCM primarily demonstrates feasibility of permit-based arbitration for intra-process runtime composition. In contrast, this thesis focuses on:

- Dynamic adaptation of OpenMP team sizes
- Runtime-level modification of the OpenMP execution model
- Fine-grained scaling decisions based on runtime feedback
- Potential extension toward inter-process and node-wide coordination

Thus, TCM serves as an architectural reference and a validation of the permit abstraction, while the primary contribution of this thesis lies in designing and implementing dynamic resource management mechanisms directly inside the OpenMP runtime. In summary, TCM illustrates that coordinated arbitration can resolve oversubscription issues within a single application. Building upon this insight, this thesis investigates how similar principles can be embedded natively into OpenMP to achieve dynamic, adaptive, and potentially cross-process resource management.

6 Free Agent Threads in OpenMP

One of the fundamental limitations of the OpenMP programming model is its rigid fork–join execution structure. Although tasking was introduced to increase flexibility, task execution is still largely bound to the lifetime and structure of parallel regions. As a result, OpenMP applications often fail to fully exploit available hardware resources in scenarios with nested parallelism, phase-based parallelism, or hybrid MPI+OpenMP execution, where load imbalance across regions or processes leads to idle CPU cores.

To address this limitation, López et al. propose the concept of free agent threads, an extension to the OpenMP execution model aimed at increasing malleability, i.e., the ability of an application to dynamically adapt its resource usage at runtime [Lop+21]. Free agent threads are OpenMP threads that are explicitly not part of any parallel team. Instead of participating in work-sharing constructs or team-level synchronization, their sole purpose is to execute explicit OpenMP tasks. This design allows free agent threads to opportunistically utilize idle computational resources by stealing tasks from active task teams, even across different parallel regions.

A key design decision is that free agent threads are not associated with OpenMP teams. This avoids semantic ambiguities related to barriers and team-level synchronization, while enabling dynamic creation, suspension, and destruction of free agent threads during program execution. Tasks eligible for execution by free agent threads must be explicit and untied, ensuring that they can be safely migrated between worker threads. To mitigate correctness issues in existing OpenMP codes that implicitly assume tasks are executed only by team threads (e.g., reliance on `omp_get_thread_num()` or `threadprivate` data), the authors introduce a proposed `free_agent` clause for task constructs, allowing developers to explicitly control whether tasks may be executed by free agent threads [Lop+21].

The authors implement a prototype of free agent threads in the LLVM OpenMP runtime, requiring approximately 800 lines of code changes. In this implementation, free agent threads are realized as operating system threads that steal tasks from the task queues of regular OpenMP threads. The runtime exposes control mechanisms via environment variables, runtime library routines, and the OMPT interface, enabling external resource managers to dynamically enable or disable free agent threads.

The effectiveness of the approach is demonstrated through two realistic HPC use cases. In a pure OpenMP application with nested parallel regions derived from the

DMRG++ code, free agent threads reduce intra-process load imbalance and achieve speedups of up to 36%. In a hybrid MPI+OpenMP application based on the Alya CFD code, free agent threads are coordinated by the Dynamic Load Balancing (DLB) library via OMPT callbacks to mitigate inter-process load imbalance, resulting in performance improvements of 10–20% [Lop+21].

Overall, free agent threads represent a practical and low-overhead mechanism to introduce dynamic resource malleability into OpenMP. By decoupling task execution from parallel region boundaries, they enable efficient utilization of shared-memory resources and provide a foundation for tighter integration between OpenMP runtimes and external resource management systems.

7 Custom OpenMP Resource Manager

This chapter describes the design and implementation of a custom dynamic resource manager (DRM) for OpenMP and documents the integration challenges encountered during development. The resource manager is implemented as a standalone process that communicates with a modified LLVM OpenMP runtime via a Unix domain socket. It intercepts thread allocation decisions at the fork barrier and applies configurable scheduling policies to control the number of threads granted to concurrent OpenMP applications.

7.1 Architecture and Integration

The resource manager is implemented in Rust as a separate process, decoupled from the OpenMP runtime. Communication between the runtime and the manager uses a Unix domain socket, following a lightweight request–reply protocol: the runtime sends a fixed-size request message describing the parallel region (including the requested thread count, a hint, and a maximum), and the manager responds with the granted thread count.

Within the modified `libomp`, the integration point is the thread allocation path that determines the final team size before a parallel region is forked. When `OMP_DYNAMIC=true`, the runtime queries the resource manager instead of applying its default heuristics. The response is cached locally in the runtime with a short TTL to amortize socket round-trip overhead for high-frequency parallel regions.

The resource manager follows the strategy pattern: a common scheduler interface is implemented by multiple interchangeable scheduling algorithms. This separation ensures that different resource management policies can be evaluated without modifying the core communication infrastructure. The following strategies were implemented:

- **Static allocation:** A fixed thread count is granted to all clients, regardless of load or contention. This serves as a controlled baseline.
- **Fair-share (reactive):** Available CPU capacity is divided equally among currently connected clients. Thread counts are recomputed on each request based on the number of active clients.

- **Predictive (proactive):** A lightweight performance model is used to anticipate future thread demand and pre-allocate resources before contention occurs.

7.2 Implementation Challenges

Three non-obvious issues were encountered during implementation and debugging, each illustrating a different failure mode at the boundary between the DRM, the OpenMP runtime, and the operating system. Resolving these issues was necessary to obtain meaningful experimental results and provides practical insights into the complexity of runtime-level resource management.

7.2.1 Fair-Share Scheduler Override

The first issue was located in the fair-share scheduler implementation (`fair_share.rs`). After correctly computing a reduced fair-share grant based on the number of active clients, the scheduler applied an unconditional hint-boost that overrode the computed result:

```
// Bug: overwrites the fair-share grant with the client's hint
granted = granted.max(req.hint_threads.min(req.max_threads.max(1)));
```

With two clients each requesting 32 threads on a system with a total capacity of 32, the fair-share computation correctly produced a grant of 16 per client. However, the hint-boost immediately raised each grant back to 32, resulting in 64 threads competing for 32 cores. The consequence was severe OS-level thread contention: FT and CG executed approximately twice as slowly as expected, with high runtime variance.

The fix was to remove the hint-boost line entirely. The fair-share grant is now final and unconditional, and the observed variance disappeared after this correction.

7.2.2 LLVM Runtime Load-Average Heuristic Override

The second issue was located in the LLVM OpenMP runtime itself (`kmp_runtime.cpp`). When `OMP_DYNAMIC=true`, the runtime performs two independent thread-count adjustments per parallel region:

1. A query to the external resource manager — returning the correct fair-share grant.
2. An internal call to `__kmp_reserve_threads()` operating in `dynamic_load_balance` mode, which reads `/proc/loadavg` and independently reduces the team size to 1 if the system appears loaded.

Because both concurrent worker processes were running threads on the same node, the system load average exceeded the threshold consistently. As a result, the load-balance heuristic silently overrode the DRM grant and serialized every EP parallel region to a single thread. EP at 32 requested threads ran in approximately 112 seconds — matching the single-threaded runtime — instead of the expected 3–4 seconds.

The fix was to set `KMP_DYNAMIC_MODE=thread_limit` in the benchmark environment. This disables the load-average heuristic and delegates full thread-count authority to the DRM. After this fix, EP performance returned to expected values.

This issue exposes a subtle but significant limitation of the current OpenMP implementation: `OMP_DYNAMIC=true` activates multiple independent dynamic mechanisms that can interfere with one another, and there is no standardized way for an external resource manager to assert exclusive control over thread allocation. This observation is discussed further in Chapter 9.

7.2.3 Sequential Benchmark Iterations

The third issue was in the benchmark script (`test-one.sh`). Benchmark iterations were executed sequentially rather than concurrently. As a result, the DRM never observed more than one active client at a time and always granted full capacity. The multi-process contention scenario — the primary use case for which the DRM was designed — was never actually exercised in early experiments.

The fix was to launch benchmark iterations in parallel (using shell background execution and `wait`) so that two processes compete simultaneously for DRM-managed resources. This is the configuration used in all results presented in Chapter 8.

7.3 CPU Affinity and Exclusive Partitioning

After resolving the three bugs above, a further structural issue with fairness was identified when using thread-count-based fair-share together with `OMP_PROC_BIND=close`.

With `OMP_PROC_BIND=close`, the OpenMP runtime pins threads to the first N consecutive cores in the process's CPU affinity set. When the DRM grants each of two concurrent processes 16 threads out of 32 available cores, both processes use the default affinity set and therefore map all their threads to the same first 16 cores, leaving the remaining 16 cores idle. The consequence is that half the available memory bandwidth goes unused, and both processes compete for the same physical cores despite the fair-share grant.

The root cause is that thread-count-based fair-share cannot prevent pinning collisions: granting each process 16 threads does not control which 16 cores those threads are mapped to.

The solution adopted in the final evaluation is explicit CPU-set partitioning via `taskset`. The available CPU domain is split in half and each concurrent process is assigned an exclusive subset of cores. The DRM capacity is set to the total thread budget t, so fair-share grants each of the two clients $t/2$ threads, which exactly fills their exclusive $t/2$-core partition. This eliminates inter-process cache and bandwidth contention and allows the DRM's effect to be measured in isolation from pinning artifacts.

This finding has a direct implication for standardization: a thread-count interface alone is insufficient for correct fair-share scheduling in affinity-aware workloads. A complete resource management interface must be able to grant exclusive CPU sets, not merely thread counts. This limitation of the current design is revisited in Chapter 9.

8 Evaluation and Benchmarking

This chapter evaluates the custom OpenMP resource manager using the NAS Parallel Benchmarks. The evaluation is structured in two parts. First, single-process baseline measurements on a local development machine characterize the performance of each benchmark kernel under static thread configurations. Second, a concurrent two-process evaluation on the LRZ CoolMUC-4 cluster quantifies the benefit of DRM-managed exclusive CPU partitioning over naive oversubscription.

8.1 Experimental Setup

8.1.1 Hardware Platforms

Two hardware environments were used across the evaluation. Local baseline measurements were performed on a single-socket machine with an Intel Core Ultra 9 285H, 16 physical cores, 16 hardware threads with simultaneous multithreading (SMT) disabled, and a single NUMA domain. This configuration provides a practical upper bound for efficient single-application parallelism at 16 threads.

The concurrent DRM evaluation was performed on a node of the LRZ CoolMUC-4 cluster. All results in Section 8.4 use this environment. Experiments were conducted with a total thread budget $t \in \{1, 4, 8, 16, 32\}$, with two concurrent processes each receiving an exclusive half of the available CPU set (see Subsection 8.1.2).

8.1.2 Evaluation Methodology

The concurrent evaluation compares two configurations:

- **dyn=true:** The OpenMP runtime queries the DRM. The DRM grants each of the two concurrent processes $t/2$ threads on their exclusive $t/2$-core partition. Thread counts are right-sized to available cores; no oversubscription occurs.
- **dyn=false:** The DRM is bypassed. Each process uses all t threads on its $t/2$-core exclusive partition, resulting in a factor-of-2 oversubscription within each partition.

Explicit CPU-set partitioning is enforced via `taskset` so that both configurations share exactly the same core assignment. This ensures that any performance difference is attributable solely to thread-count matching rather than to core placement. The environment variable `KMP_DYNAMIC_MODE=thread_limit` is set in all runs to disable the LLVM runtime's internal load-average heuristic, which would otherwise silently override the DRM grant (see Section 7.2). All results are averages over 20 measurements.

8.2 NAS Parallel Benchmarks

The NAS Parallel Benchmarks (NPB) are a widely used suite of compact kernels intended to represent characteristic computation and communication patterns found in scientific workloads []. NPB provides a standardized baseline that helps relate the observed effects of resource management strategies to well-known computational behaviors.

8.2.1 Kernel Suite

NPB comprises five kernels: IS (Integer Sort; random memory access), EP (Embarrassingly Parallel), CG (Conjugate Gradient; irregular memory access and global reductions), MG (Multi-Grid; long- and short-distance communication with high memory intensity), and FT (discrete 3D Fast Fourier Transform; all-to-all communication) [].

8.2.2 Selection Rationale

The detailed analysis focuses on EP, CG, and FT. Together, these kernels cover complementary extremes that are particularly relevant for dynamic resource management: EP is dominated by independent computation with minimal synchronization; CG represents sparse linear algebra with irregular access patterns and global reductions; and FT is characterized by communication-heavy phases with global data rearrangement [JFY].

8.2.3 EP (Embarrassingly Parallel)

The EP kernel is a Monte Carlo-style benchmark. It generates independent pairs of Gaussian random numbers and bins them into concentric square annuli, with communication limited to a final global reduction [JFY]. EP is a useful baseline for policies under low contention for shared data, where performance is expected to be primarily sensitive to available compute resources rather than memory bandwidth or synchronization overhead.

8.2.4 CG (Conjugate Gradient)

CG models the solution of a large sparse linear system. The benchmark applies the inverse power method with the Conjugate Gradient algorithm to estimate the largest eigenvalue of a symmetric positive definite sparse matrix with a random nonzero pattern [JFY]. It stresses irregular memory access and collective synchronization, making it sensitive to both thread-level parallelism and memory hierarchy effects.

8.2.5 FT (Fast Fourier Transform)

FT is a 3D FFT-based PDE kernel. It numerically solves a partial differential equation by alternating forward and inverse Fourier transforms, producing phases with substantial global data movement such as matrix transposes [JFY]. FT is well-suited for studying how thread count changes interact with memory bandwidth and synchronization bottlenecks.

8.3 Single-Process Baseline

To establish reference performance for individual benchmark kernels, static configurations were measured on the local 16-core machine without concurrent processes or DRM involvement. All benchmarks use NPB Class C.

For CG, performance peaks at 16 threads with a runtime of approximately 66 s and a throughput of approximately 2160 Mop/s. Increasing the thread count beyond 16 degrades performance due to oversubscription: context switching overhead and cache contention reduce effective throughput. For FT, 16 threads achieves approximately 34 s and 11 650 Mop/s. For IS (Integer Sort, four concurrent instances), static scheduling at 16 threads per instance achieves a total throughput of approximately 2270 Mop/s with a fairness ratio (minimum to maximum instance throughput) of 0.94, indicating balanced load distribution.

These baselines confirm that, for all kernels, the optimal single-process thread count matches the number of available physical cores, and oversubscription is harmful in all cases [Ian+10].

8.4 DRM Evaluation: Concurrent Two-Process Results

The following results compare dyn=true (DRM-managed, no oversubscription) against dyn=false (oversubscribed, no DRM) for two processes running concurrently with exclusive CPU-set partitioning.

8.4.1 EP (Embarrassingly Parallel)

Table 8.1: EP (Class C) runtime for two concurrent processes with exclusive CPU partitioning. t denotes the total thread budget; each process receives $t/2$ exclusive cores.

t	dyn=true (s)	dyn=false (s)	Speedup
1	56.1	56.1	—
4	28.1	28.1	≈0%
8	14.0	14.1	+1%
16	7.0	7.2	+3%
32	3.7	3.8	+3%

EP shows minimal sensitivity to oversubscription. Because EP is compute-bound with no shared data structures beyond the final reduction, the OS time-sharing overhead of doubling the thread count on a fixed core set is modest. The DRM's benefit is correspondingly small: approximately 1–3% at higher thread counts. EP therefore represents a best case for oversubscription and a worst case for the DRM's contribution.

8.4.2 FT (3-D FFT, Memory-Bandwidth Bound)

Table 8.2: FT (Class C) runtime for two concurrent processes with exclusive CPU partitioning.

t	dyn=true (s)	dyn=false (s)	Speedup
1	104.6	105.5	+1%
4	52.7	53.5	+2%
8	26.5	27.3	+3%
16	13.8	14.4	+4%
32	7.8	8.8	**+12%**

FT exhibits a clearer DRM benefit, peaking at +12% for $t = 32$. FT involves phases of intensive global data movement (3D transposes). When the thread count is doubled on the same core set, cache line contention and memory bandwidth saturation increase non-linearly. The DRM's right-sizing of threads to cores reduces these effects and produces lower, more stable runtimes.

8.4.3 CG (Conjugate Gradient, Irregular Memory Access)

Table 8.3: CG (Class C) runtime for two concurrent processes with exclusive CPU partitioning.

t	dyn=true (s)	dyn=false (s)	Speedup
1	91.7	94.7	+3%
4	45.5	47.7	+5%
8	23.4	24.4	+4%
16	12.4	13.8	+11%
32	7.4	9.3	**+26%**

CG shows the strongest DRM benefit, reaching +26% at $t = 32$. CG applies the conjugate gradient algorithm to a sparse matrix with a randomized nonzero structure, producing highly irregular memory access patterns. These access patterns stress the cache hierarchy and DRAM bandwidth. When two CG instances each run twice as many threads as available cores, the resulting cache pressure and TLB thrashing are severe. Right-sizing threads to cores, as the DRM enforces, reduces contention at every level of the memory hierarchy.

8.5 Key Findings

The results across all three benchmarks support the following conclusions:

1. The DRM is faster at every data point. There is no configuration where dyn=false outperforms dyn=true. The DRM's overhead — one Unix socket round-trip per parallel region, cached with a short TTL — is consistently below measurement noise.

2. The benefit scales with thread count. At low t (1–4), oversubscription involves few extra threads and OS scheduling handles the contention cheaply. At $t = 32$, each process oversubscribes its exclusive partition by a factor of two (32 threads on 16 cores), and the performance penalty becomes substantial: +12% for FT, +26% for CG.

3. Benefit correlates with memory access pattern. EP (compute-bound, independent) benefits minimally (+3%). FT (bandwidth-bound, structured) benefits moderately (+12%). CG (irregular sparse access, high cache pressure) benefits most (+26%). This ordering matches the expectation from performance modeling [Amd67]: kernels with high data reuse and shared memory pressure are penalized most severely by oversubscription.

4. The DRM reduces runtime variance. Without the DRM, dyn=false maximum runtimes are 30–50% higher than averages due to non-deterministic OS scheduling under

oversubscription. With the DRM, variance is consistently low across all benchmarks. This stability improvement may be as important as the mean speedup in production HPC workloads, where worst-case latency bounds matter for job scheduling [YJG03].

9 Implications and Standardization Proposal for OpenMP

10 Summary

Abbreviations

Bibliography

[] *NAS Parallel Benchmarks*.

[] *The LLVM Compiler Infrastructure Project*.

[] *Welcome to the documentation of OpenMP in LLVM! — LLVM/OpenMP documentation*.

[Ali+22] J. I. Aliaga, M. Castillo, S. Iserte, I. Martín-Álvarez, and R. Mayo. "A Survey on Malleability Solutions for High-Performance Distributed Computing." In: *Applied Sciences* 12.10 (2022), p. 5231. DOI: 10.3390/app12105231.

[Amd67] G. M. Amdahl. "Validity of the single processor approach to achieving large scale computing capabilities." In: *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference*. AFIPS '67 (Spring). New York, NY, USA: ACM, 1967, pp. 483–485. DOI: 10.1145/1465482.1465560.

[DAm+18] M. D'Amico, M. Garcia-Gasulla, V. López, A. Jokanovic, R. Sirvent, and J. Corbalan. "DROM: Enabling Efficient and Effortless Malleability for Resource Managers." In: *Proceedings of the 47th International Conference on Parallel Processing Companion*. Eugene OR USA: ACM, Aug. 2018, pp. 1–10. ISBN: 978-1-4503-6523-9. DOI: 10.1145/3229710.3229752.

[DM98] L. Dagum and R. Menon. "OpenMP: an industry standard API for shared-memory programming." In: *IEEE Computational Science and Engineering* 5.1 (Mar. 1998), pp. 46–55. ISSN: 10709924. DOI: 10.1109/99.660313.

[Eic+12] A. E. Eichenberger, C. Terboven, M. Wong, and D. an Mey. "The Design of OpenMP Thread Affinity." In: *OpenMP in a Heterogeneous World, 8th International Workshop on OpenMP, IWOMP 2012*. Vol. 7312. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2012, pp. 15–28. DOI: 10.1007/978-3-642-30961-8_2.

[Eic+13] A. E. Eichenberger, J. M. Mellor-Crummey, M. Schulz, M. Wong, N. Copty, R. Dietrich, X. Liu, E. Loh, and D. Lorenz. "OMPT: An OpenMP Tools Application Programming Interface for Performance Analysis." In: *Using and Improving OpenMP for Devices, Tasks, and More*. Vol. 8122. Lecture Notes in Computer Science. Heidelberg: Springer, 2013, pp. 171–185. DOI: 10.1007/978-3-642-40698-0_13.

[Fec] J. Fecht. "A Simulation Layer for Dynamically Varying Computing Resources in MPI." In: ().

[FR96] D. G. Feitelson and L. Rudolph. "Toward Convergence in Job Schedulers for Parallel Supercomputers." In: *Job Scheduling Strategies for Parallel Processing*. Vol. 1162. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 1996, pp. 1–26. DOI: 10.1007/BFb0022284.

[FVI] A. Fedotov, M. Voss, and I. Isaev. "A Case Study for Resolving Composability Issue Using a Shared CPU Resource Coordinator." In: ().

[GLC14] M. Garcia, J. Labarta, and J. Corbalan. "Hints to improve automatic load balancing with LeWI for hybrid applications." In: *Journal of Parallel and Distributed Computing* 74.9 (2014), pp. 2781–2794. DOI: 10.1016/j.jpdc.2014.05.004.

[Gus88] J. L. Gustafson. "Reevaluating Amdahl's law." In: *Communications of the ACM* 31.5 (May 1988), pp. 532–533. DOI: 10.1145/42411.42415.

[Ian+10] C. Iancu, S. Hofmeyr, F. Blagojevic, and Y. Zheng. "Oversubscription on Multicore Processors." In: *24th IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2010, pp. 1–11. DOI: 10.1109/IPDPS.2010.5470434.

[JFY] H. Jin, M. Frumkin, and J. Yan. "The OpenMP Implementation of NAS Parallel Benchmarks and Its Performance." In: ().

[LA04] C. Lattner and V. Adve. "LLVM: A compilation framework for lifelong program analysis & transformation." In: *International Symposium on Code Generation and Optimization, 2004. CGO 2004.* San Jose, CA, USA: IEEE, 2004, pp. 75–86. ISBN: 978-0-7695-2102-2. DOI: 10.1109/CGO.2004.1281665.

[Lop+21] V. Lopez, J. Criado, R. Peñacoba, R. Ferrer, X. Teruel, and M. Garcia-Gasulla. "An OpenMP Free Agent Threads Implementation." In: *OpenMP: Enabling Massive Node-Level Parallelism*. Ed. by S. McIntosh-Smith, B. R. De Supinski, and J. Klinkenberg. Vol. 12870. Lecture Notes in Computer Science. Cham: Springer International Publishing, 2021, pp. 211–225. ISBN: 978-3-030-85261-0. DOI: 10.1007/978-3-030-85262-7_15.

[Men04] P. Menage. *Control Groups*. Linux kernel documentation. The Linux Kernel Archives. 2004.

[Ope24] OpenMP Architecture Review Board. *OpenMP Application Programming Interface Version 6.0*. Tech. rep. Available at https://www.openmp.org/specifications/. OpenMP Architecture Review Board, Nov. 2024.

[Pro+15] J. Protze, I. Laguna, D. H. Ahn, J. DelSignore, A. Burton, M. Schulz, and M. S. Müller. "Lessons Learned from Implementing OMPD: A Debugging Interface for OpenMP." In: *OpenMP: Heterogenous Execution and Data Movements*. Vol. 9342. Lecture Notes in Computer Science. Cham: Springer, 2015, pp. 89–101. DOI: 10.1007/978-3-319-24595-9_7.

[Sch] M. Schreiber. "Cluster-Based Parallelization of Simulations on Dynamically Adaptive Grids and Dynamic Resource Management." In: ().

[SW18] M. Schreiber and T. Weinzierl. "A Case Study for a New Invasive Extension of Intel's Threading Building Blocks." In: (Jan. 2018), pp. 21–26. DOI: 10.14459/2018MD1428538.

[YJG03] A. B. Yoo, M. A. Jette, and M. Grondona. "SLURM: Simple Linux Utility for Resource Management." In: *Job Scheduling Strategies for Parallel Processing, JSSPP 2003*. Vol. 2862. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2003, pp. 44–60. DOI: 10.1007/10968987_3.